

Gestión de Vulnerabilidades

Seguridad en la Cadena de Suministro

Producción de Software EMI305

Nombres: Diego Marillán , Danko Torres

profesor: Dr. Pablo Valenzuela

| CONTEXTO Y ALCANCE

22

REPOSITORIOS ANALIZADOS CON EVIDENCIA

El análisis se centró en la pregunta estratégica: ¿**Cómo gestionar vulnerabilidades considerando su origen y nivel de riesgo real?**

 **Motores usados:** SBOM, Grype, CodeQL y CI/CD Auditor.

 **Scripts Propios:** Extracción y transformación de datos brutos a métricas accionables.



RADIOGRAFÍA DEL RIESGO

878

HALLAZGOS CODEQL

5 Críticos (P0) / 188 Prioritarios (P1)

570

RIESGOS CI/CD

11 Críticos (P0) - Permisos Write & Secretos

570

ALERTAS GRYPE

100% Severidad Baja (Ruido masivo)

Conclusión del Escaneo

Existe una disparidad crítica entre el volumen y el peligro real. Mientras las dependencias (Grype) generan el mayor ruido acumulativo, las vulnerabilidades explotables se concentran en el **código fuente** y la configuración de **pipelines**, sumando un total de 16 flujos críticos (P0).



Nota: El 100% de los hallazgos Grype son de severidad baja. El riesgo crítico reside en Código Fuente y CI/CD.

| REPOSITARIOS CRÍTICOS



opencode

Prioridad P0. Inyección de comandos dinámica y alto riesgo en CI/CD.



Understand-Anything

Prioridad P0. Shell injection desde el entorno. Crítico para la cadena.



ruflo / n8n-mcp

Prioridad P2. Alto volumen de dependencias pero impacto directo bajo.

** Se analizaron 22 repositorios en total, incluyendo Anthropic-Skills y maigret (P1).*

| VECTORES DE ATAQUE

Código & Dependencias

Foco: `js/command-line-injection`.

Vulnerabilidades en el flujo de datos que permiten ejecución arbitraria.

Pipelines CI/CD

Foco: `pull_request_target` e `id-token: write`.

Permisos excesivos y secretos expuestos en flujos de automatización.

Factor Humano

Foco: **Falta de Triage**.

Políticas ausentes en la revisión de flujos y gestión de secretos.

CICLO: DEL DATO A LA ACCIÓN



Conozco

Detección automática via CodeQL/Grype.

Verifico

Análisis humano del flujo y contexto.

Evidencio

Registro inmutable en reportes CSV/MD.

Actúo

Priorización y mitigación táctica.



01. CONOZCO

Detección Inteligente

Identificación de telemetría inicial mediante motores automatizados.

- Escaneo de SAST (CodeQL)
- Análisis de SCA (Grype)
- Inventario de SBOM
- Auditoría de CI/CD



02. VERIFICO

Contextualización Análisis humano para validar la explotabilidad real del hallazgo.

- Revisión de flujo de datos
- Descarte de falsos positivos
- Evaluación de alcance (Prod)
- Análisis de vectores



03. EVIDENCIO

Registro Inmutable

Transformación de alertas en artefactos técnicos de auditoría.

- Generación de CSV/Markdown
- Trazabilidad de líneas
- Logs de configuración
- Consolidación en reportes



04. ACTÚO

Mitigación Táctica Ejecución de la decisión basada en la prioridad de riesgo asignada.

- Bloqueo de Release (P0)
- Refactorización de código
- Aceptación de riesgo (P3)
- Hardening de Workflows

EVIDENCIA Y TRAZABILIDAD

Artefacto de Evidencia	Propósito Estratégico
codeql_findings.md	Trazabilidad exacta de reglas PO y líneas de código vulnerables.
cicd_risks.csv	Identificación de workflows con acceso a secretos sin protección.
grype_findings.csv	Inventario de vulnerabilidades en dependencias (Grype Low).
repositories_summary.md	Visión ejecutiva del ecosistema y componentes detectados.

| MATRIZ DE PRIORIZACIÓN

P0: MITIGAR HOY

Riesgo: Inyección de comandos y abuso de CI/CD.

Repos: opencode, Understand-Anything.

P1: CORTE PLAZO

Riesgo: Fuga de secretos en logs, XSS DOM.

Repos: Anthropic-Skills, claude-code.

P2: PLANIFICADO

Riesgo: Deuda técnica, dependencias bajas.

P3: MONITOREO

Riesgo: Sin hallazgos de seguridad activos.

PLAN DE MITIGACIÓN: ACCIONES PROPUESTAS

⚠️ 01. INTERVENCIÓN PO

Remediación Inmediata:

Corregir las vulnerabilidades críticas de inyección de comandos detectadas de manera prioritaria.

Acción restrictiva: Bloqueo absoluto de nuevos *releases* si los workflows comprometidos son alcanzables desde el exterior.

Repositorios foco: opencode y Understand-Anything.

🛡️ Aislar pull_request_target

🔑 Reducir Permisos en CI/CD

🔒 Hardening por SHA Completo

🔍 Higiene de Secretos

🔄 Parcheo de Dependencias

👥 Gobierno: Triage Activo

Hallazgos Principales

P0

Riesgo: Inyección de comandos

```
const command = `run ${userInput}`; // Concatenación insegura
```

Repos: opencode, Understand-Anything

P1

Riesgo: Logging y almacenamiento de datos sensibles y hashing débil.

Repos: Anthropic-Skills, claude-code.

P1

Riesgo: Log injection y hashing débil; logging de datos sensibles

Repos: maigret, jcode

P2

Riesgo: Alto volumen de vulnerabilidades Gripe bajas.

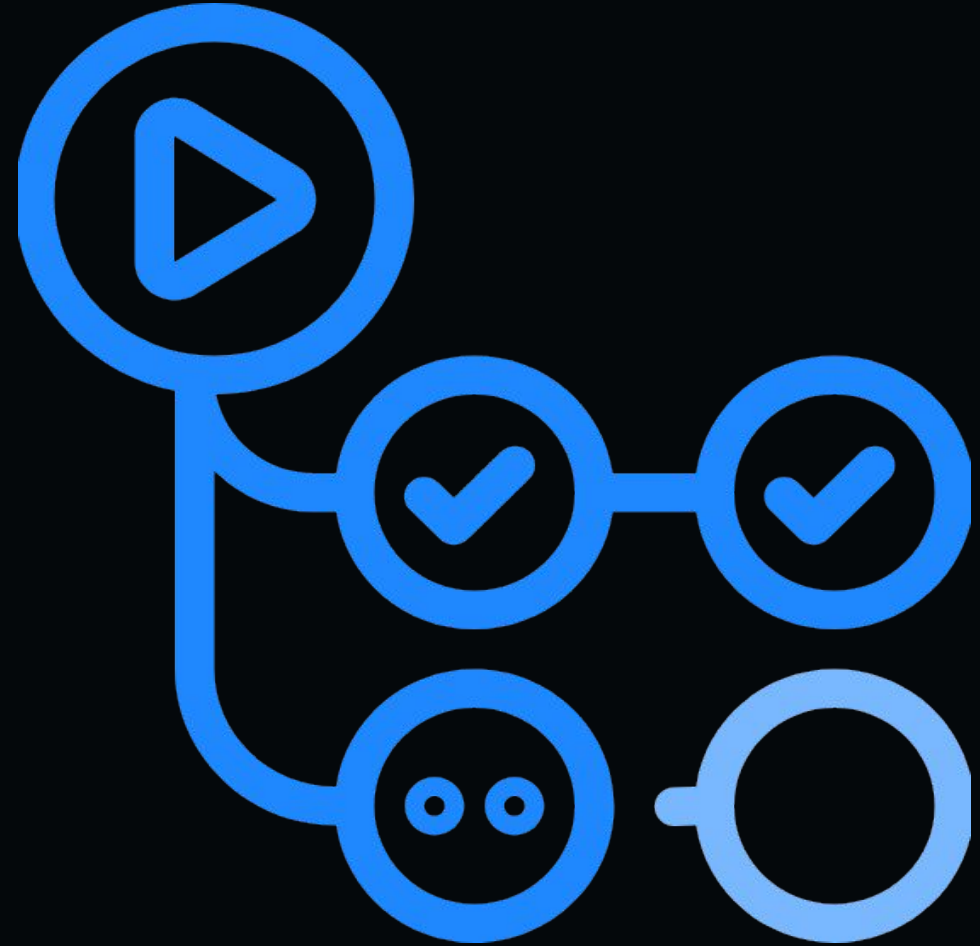
Repos: ruflo, n8n-mcp, presenton, TradingAgents

| INTERVENCIÓN TÉCNICA

🚫 **Bloqueo:** Detener despliegues si flujos P0 son alcanzables externamente.

</> **Refactor:** Reemplazar comandos dinámicos por APIs robustas.

🔒 **Pinning:** Fijar GitHub Actions por SHA completo para evitar inyecciones.



| EL FACTOR HUMANO

Triage SLA

Establecer tiempos de respuesta:
PO en 24-48h, P1 en una semana.

Code Review

Revisión obligatoria para cualquier
cambio en `.github/workflows/`.

Secret Mgmt

Justificación documentada para el
uso de secretos en workflows.



- ✓ El proceso humano es lo único que mitiga el riesgo detectado por la herramienta.
- ✓ **Política de Triage:** La única remediación verdadera para el volumen masivo.
- ✓ Eliminación de secretos en automatizaciones estándar y estandarización de revisiones.

570

RIESGOS CI/CD

878

ALERTAS CODEQL

GRACIAS